The Faculty of Science and Engineering

Department of Electrical and Electronic Engineering



EEEE4077 Mechatronics Project
Project Outline and Planning Report

Vision-Augmented Mobile Robot Navigation in Dynamic Environments

AUTHOR	:	Nicholas Khor Han Le
ID	:	20617556
SUPERVISOR:	Dr. Shalini Darmaraju	
MODERATOR	:	Ir Dr. Daniel Pu
DATE	:	28 October 2026

Statement of Originality

University of Nottingham | Faculty of Science and Engineering | Department of Electrical and Electronic Engineering | EEEE4077 Mechatronics Project

- I have read and understood the University of Nottingham Academic Misconduct Policy.
- I am aware that failure to act in accordance with the Regulations Governing Academic Integrity may lead to the imposition of penalties which, for the most serious cases, may include termination of programme.
- I consent to the University copying and distributing any or all of my work in any form and using third parties to verify whether my work contains plagiarised material, and for quality assurance purposes.

Note: *You must change the statements in the boxes to reflect your individual submission.*

1. Acknowledgment of Sources

You must acknowledge all sources of information (e.g. ideas, algorithms, data) using citations to auditable, authoritative sources. You must also put quotation marks around any sections of text that you have copied without paraphrasing and cite the original source. If any figures or tables have been taken or modified from another source, you must explain this in the caption, and cite the original source.

I have acknowledged all sources used in this report and cited them using IEEE numbering [1]–[13]. No text has been copied without attribution, and the block diagrams, flowcharts, and schedule tables were prepared by me for this project.

2. Resources Produced by Someone Else

If you have used any code (e.g., open-source code), reference designs, or similar resources that have been produced by anyone else, you must list them in the box below. In the report, you must explain what was used and how it relates to the work you have done.

I plan to use the following open-source software and libraries produced by others:

1. ROS 2 (Jazzy / Humble) and the Nav2 stack (nav2_mppi_controller, nav2_costmap_2d) for robot communication, coordinate frames (tf2), and the baseline local planner.
2. Gazebo Harmonic for the indoor corridor simulation, differential-drive robot physics, simulated sensors, and scripted moving obstacle actors.
3. Ultralytics YOLOv8 (pre-trained yolov8n weights) and ByteTrack (boxmot) for 2D bounding box detection and frame-to-frame object tracking.
4. Standard open-source math and vision libraries (OpenCV, Eigen3, NumPy, SciPy, Matplotlib) for matrix calculations and data plotting.

The camera-LiDAR frustum depth matching, linear Kalman filter velocity tracker, custom C++ MPPI dynamic obstacle critic plugin, and automated testing scripts will be written by me.

3. Generative AI Declaration

If you have used generative AI tools in the preparation of your assessed work, or the underlying code/designs/underpinning work, its use must be declared in a signed statement in the box below. You must declare the tools employed and the method and extent of use. You can do this using the table below:

Generative AI tools have been used in the preparation of this work as follows:

Tool	Application Scope	Nature of Assistance
ChatGPT / Google Gemini	Grammar checking and syntax lookup	Used for checking grammar and phrasing in draft sections, and for looking up standard ROS 2 / C++ syntax and configuration boilerplate.

I confirm that all project design decisions, parameter choices, and final written sections are my own work.

4. Independent Work & External Assistance

You can consult with teaching staff/demonstrators, but you should not show anyone else your work (this includes uploading your work to publicly accessible repositories e.g. GitHub, unless expressly permitted by your supervisor). Nor should you help other students with their work. You must get permission in writing before you seek outside assistance, e.g. a proofreading service, and declare it here.

I completed this work myself and have not shared my work with other students or used any external proofreading services. My project repository is kept private.

5. Research Data Integrity & Reproducibility

You must not have fabricated, modified or distorted any data, evidence, references, experimental results, or other material used or presented in the report. You must clearly describe your experiments and how the results were obtained, and include all data, source code and/or designs (submitted as a separate file) so that your results could be reproduced.

The material in this planning report is genuine, and all simulation parameters and evaluation methods are designed to be reproducible in the simulation workspace.

6. Ethical Approval

If your work involved research studies (including surveys) on human participants, their cells or data, or on animals, you must have been granted ethical approval before the work was carried out, and any experiments must have followed these requirements. You must give details of this in the report, and list the ethical approval reference number(s) in the box below.

Not applicable. This project is carried out entirely in computer simulation (Gazebo Harmonic and ROS 2) using virtual obstacle actors and does not involve human participants or animals.

Supervisory Discussion & Final Sign-Off

My responses to these questions are full and complete, and I have discussed them with my supervisor.

Field	Detail
Student Name	Nicholas Khor Han Le
Student ID	20617556
Degree Programme	MEng (Hons) Electrical and Electronic Engineering / Mechatronics
Project Code & Title	SD-MEng-26-01: Vision-Augmented Mobile Robot Navigation in Dynamic Environments
Supervisor	Dr. Shalini Darmaraju
Moderator	Ir Dr. Daniel Pu
Date	2 November 2026
Student Signature	Nic

Table of Contents

Statement of Originality.....2
1. Overview of the Project.....5
1.1 Background and the Freezing Robot Problem.....5
1.2 Project Aim and Objectives.....5
1.3 System Architecture.....5
2. Specification.....7
3. Methodology.....8
3.1 Development Steps and Tools.....8
3.2 Workflow and Algorithm Choices.....8
4. Originality.....10
4.1 Project Novelty and Comparison.....10
4.2 Project Limitations.....11
5. Risk Management.....11
6. Planned Work Schedule.....12
References.....13
Appendix A: Supervisor-Provided Project Specification.....14

1. Overview of the Project

1.1 Background and the Freezing Robot Problem

According to the International Federation of Robotics (IFR), transportation and logistics made up the largest commercial segment for professional service robots in 2024, with 102,900 units sold globally (a 14% annual increase) [1]. Most of these Autonomous Mobile Robots (AMRs) are deployed in existing brownfield factories and warehouses that cannot be redesigned with dedicated robot-only lanes. Instead, the robots have to share narrow corridors with walking workers, pallet jacks, and forklifts. To keep hardware costs reasonable and meet industrial safety standards (ISO 3691-4), most commercial AMRs rely on a single-plane 2D safety laser scanner paired with the open-source Robot Operating System 2 (ROS 2) navigation stack, Nav2 [2], [3]. While a 2D LiDAR provides accurate range measurements across a 270° to 360° field of view at 20 to 40 Hz, combining a planar laser scan with a standard 2D occupancy grid (nav2_costmap_2d) causes a well-known failure mode in busy corridors called the freezing robot problem [4].

As described by Trautman and Krause [4], the freezing robot problem happens when a mobile robot encounters moving obstacles and either stops completely, oscillates in place, or triggers unnecessary recovery rotations because its local planner marks all forward paths as unsafe. In a standard 2D LiDAR setup, this happens for two practical reasons. First, a 2D laser scanner mounted 10 to 20 cm above the ground only slices across a walking person's lower legs, which show up as two small, jumping point clusters without any class label, bounding box, or velocity estimate. Second, standard Nav2 pipelines write these laser hits directly into a static 2D costmap [2], treating every occupied cell as a permanent wall at the current instant. When a person walks across a corridor at a normal speed of 1.2 to 1.4 m/s [9], the local planner cannot predict that the person will clear the path in a second or two. Instead, the robot brakes and waits until the path is completely clear, or gets stuck trying to replan around a moving target.

1.2 Project Aim and Objectives

Project Aim: To develop and evaluate in simulation a vision-augmented mobile robot navigation framework that combines simulated camera images with 2D planar range measurements to track moving obstacles, estimate their positions and velocities, and incorporate short-term motion predictions into local path planning and collision avoidance.

To meet this aim and match the project specification, the work is divided into three main objectives:

1. Obstacle Detection and Tracking (O1): To build an indoor mobile robot simulation and adapt visual detection and tracking methods to combine simulated camera images with planar range scans for obstacle position and velocity estimation.
2. Motion-Aware Local Path Planning (O2): To develop a local navigation controller that uses short-term obstacle motion predictions and expanding safety margins during trajectory evaluation to reduce robot freezing while avoiding collisions.
3. Comparative Simulation Evaluation (O3): To evaluate the proposed system against range-sensor-only navigation and non-predictive visual detection baselines across different obstacle speeds, movement patterns, and crowd densities, measuring tracking error, navigation success, waiting time, and computation time.

1.3 System Architecture

Figure 1 shows the top-down block diagram of the proposed system. The main design idea is to separate static building walls (which stay in the standard 2D costmap) from moving obstacles

(which are tracked in Objective O1 and forward-projected inside the local planner in Objective O2).

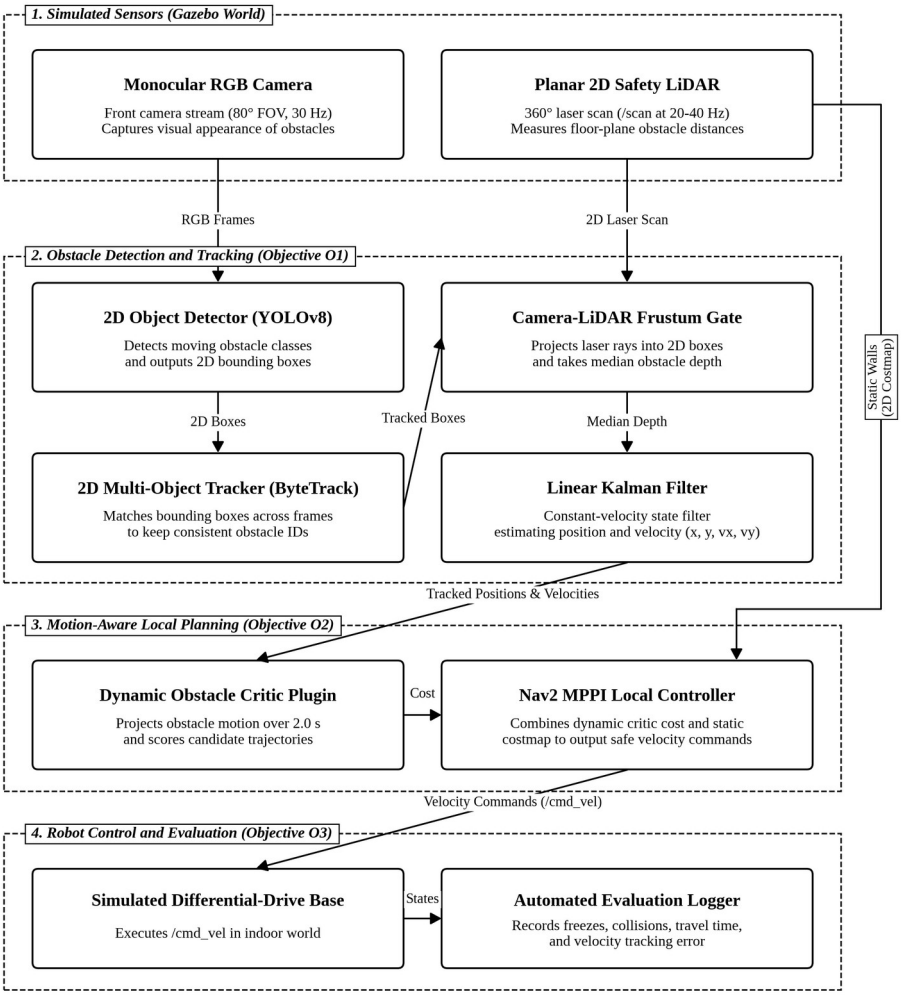


Figure 1. Top-down system architecture showing simulated sensors, obstacle tracking (O1), motion-aware local planning (O2), and evaluation logging (O3).

As shown in Figure 1, the simulated robot uses a forward-facing RGB camera (1280×720 resolution at 30 Hz, 80° horizontal field of view) and a 360° planar 2D LiDAR running at 20 to 40 Hz. In the perception pipeline (O1), an off-the-shelf YOLOv8 detector finds moving obstacle classes (such as pedestrians and carts) in each camera frame in about 12 to 18 ms. These 2D bounding boxes are passed to ByteTrack [8], which matches boxes across frames in 1 to 2 ms on the CPU so that obstacle IDs are not lost during brief partial occlusions (up to 1.0 s). Using the camera-to-LiDAR calibration method from Zhang and Pless [6], the 2D laser scan rays are projected into the camera frame. Rays falling inside each tracked 2D bounding box are filtered using the median range value so that laser rays slipping past a person's legs onto background walls are ignored. A linear Kalman filter [7] with a constant-velocity motion model then smooths these floor positions at 30 Hz (< 0.2 ms CPU time) to estimate each obstacle's 2D position and velocity vector.

In the local planning stage (O2), the tracked obstacle states are sent to a custom C++ critic plugin inside the Nav2 Model Predictive Path Integral (MPPI) controller [5], [10] running at 20 Hz. Instead of writing moving obstacles into the 2D costmap where they leave lagging raycast trails, the custom critic projects each obstacle's position forward over a 2.0 s lookahead window with a slightly expanding safety radius. This allows the MPPI planner to smoothly slow down

or steer behind a crossing obstacle while the standard 2D costmap continues to handle static corridor walls across the full 360° laser scan.

2. Specification

Table 1 lists the quantitative specifications for the project, including both minimum pass thresholds and target goals, how each requirement is justified, the test method, and the expected technical challenge (Low, Medium, or High). These metrics will be tested across five indoor simulation scenarios: perpendicular corridor crossings, narrow aisle head-on passes, faster pushcarts, multi-person intersections, and a static obstacle control run.

Table 1. Project specification metrics, target thresholds, engineering justification, and verification methods.

ID	Metric	Minimum	Target	Engineering Justification	Verification Method	Ch.
O1-a	Obstacle velocity RMSE (within 8 m)	<= 0.25 m/s	<= 0.15 m/s	About 10% of normal 1.4 m/s walking speed [9]; keeps 2.0 s position prediction error under 0.3 m.	Compare Kalman filter velocity against Gazebo ground truth across 30 runs.	M
O1-b	Camera-LiDAR box matching rate	>= 85%	>= 95%	Accurate frustum depth gating [6] stops background wall hits from corrupting the velocity filter [7].	Percentage of visual detections matched to a laser cluster within 0.3 m of ground truth.	M
O2-a	Robot freezes per 10 crossings	>= 50% below C0	0 freezes	Directly addresses the freezing robot problem [4] where static planners halt or spin in traffic.	Count planner timeouts, recovery rotations, and stops longer than 2.0 s.	H
O2-b	Route completion time vs. baseline	>= 10% faster	15-25% faster	Adjusting speed early and passing behind moving obstacles avoids stop-and-wait delays.	Compare average travel time across 30 seeded trials per scenario.	M
O2-c	Collision-free navigation	0 collisions	0 collisions	Predictive path planning must not trade away basic physical safety.	Monitored via Gazebo contact sensors; any physical contact counts as a failed run.	M
O3-a	3-way ablation statistical separation	$p < 0.05$	$p < 0.01$	Separates the benefit of visual detection alone (C1) from motion prediction (C2).	Paired t-test and one-way ANOVA across C0, C1, and C2 over obstacle speeds of 0.5-2.0 m/s.	M
O3-b	Cumulative waiting time per run	>= 50% reduction	<= 5.0 s	Measures how much time the robot wastes standing still at busy intersections.	Total time robot speed is near zero (< 0.05 m/s) while a goal is active.	M
O3-c	Real-time control loop rate	>= 15 Hz	>= 20 Hz	Local controller must keep up with the 20 Hz laser scan rate for responsive avoidance [10].	Log mean execution rate of the tracker (>= 30 Hz) and MPPI loop (>= 20 Hz).	L
NF1	Docker container reproducibility	Pass	Pass	Ensures simulation benchmarks run consistently without ROS package version issues.	Verify a single launch command inside Docker reproduces the test scenarios.	L

Each core objective maps directly to a practical software deliverable. Objective O1 produces the ROS 2 vision-LiDAR perception and Kalman tracking package, verified against simulator ground truth (O1-a and O1-b). Objective O2 produces the custom C++ dynamic obstacle MPPI critic plugin (along with a costmap inflation fallback layer), verified through closed-loop crossing trials (O2-a, O2-b, and O2-c). Objective O3 produces the automated Python scenario

test runner, statistical evaluation scripts, simulation video demonstrations, and the final dissertation report comparing configurations C0, C1, and C2.

If the core deliverables are completed ahead of schedule, three prioritized stretch goals will be explored. First (S1), a dense crowd test with 4 to 6 simultaneous moving obstacles will test how well ByteTrack and the planner handle frequent visual occlusions. Second (S2), higher-speed obstacles (up to 2.5 m/s) and diagonal crossing paths will be used to find where the constant-velocity assumption starts to break down. Third (S3), the system will be profiled under a strict 4-core CPU limit to check whether the local planner still maintains its 20 Hz update rate on constrained hardware.

3. Methodology

3.1 Development Steps and Tools

The project is developed entirely in simulation using Ubuntu 24.04 LTS, ROS 2 Jazzy, and Gazebo Harmonic. To separate the effect of camera detection from motion prediction as required by the project brief, three configurations will be compared: C0 (the standard 2D LiDAR-only Nav2 baseline), C1 (camera + 2D LiDAR detection where obstacles are treated as stationary circles at the current instant without velocity prediction), and C2 (the full framework with Kalman velocity tracking and 2.0 s forward projection in MPPI). Table 2 outlines the six methodology steps (M0 to M5), the tools used, their practical limitations, and my current skill level and preparation tasks for each stage.

Table 2. Methodology development steps (M0-M5), selected software tools, limitations, and skill assessment.

Step	Core Process & Selected Tools	Tool Limitations & Project Impact	Skill Level & Preparation Tasks
M0: Setup	Ubuntu 24.04 LTS, ROS 2 Jazzy, Gazebo Harmonic, and Docker container.	Older ROS 2 versions reach end-of-life soon; using Docker avoids package conflicts across machines.	Low challenge. Already comfortable with Linux, Docker, and basic ROS 2 workspaces.
M1: Sim & C0	Gazebo indoor corridor worlds, differential-drive robot URDF, scripted pedestrian actors, and Nav2 baseline.	Default scripted actors move on fixed loops unless randomized; contact sensors need explicit setup.	Medium challenge. Write Python actor spawners with seeded speeds (0.5-2.0 m/s) and record C0 freezing runs.
M2: Tracker (O1)	YOLOv8n detector, ByteTrack [8] 2D tracker, Zhang-Pless frustum projection [6], and Linear Kalman Filter [7].	Camera FOV (80°) is narrower than the 360° LiDAR; laser rays can slip past legs and hit walls behind them.	Low challenge. Familiar with OpenCV and Kalman filtering; add median depth gating and outlier rejection.
M3: Critic (O2)	Custom C++17 DynamicObstaclesCriticPlugin integrated into nav2_mppi_controller [5], [10].	Scoring 2,000 candidate trajectories per control cycle must finish well inside the 50 ms (20 Hz) budget.	High challenge (first time writing a Nav2 C++ plugin). Build a minimal dummy critic early to test pluginlib loading.
M4: Harness (O3)	Automated Python scenario runner, Gazebo reset services, rosbag2 logger, and ground-truth evaluator.	Running 450 batch trials across C0, C1, and C2 can hang if Gazebo or Nav2 nodes fail to reset cleanly.	Medium challenge. Add process watchdog timeouts, fixed random seeds, and headless Gazebo execution.
M5: Analysis (O3)	Python evaluation and plotting scripts using NumPy, SciPy, and Matplotlib.	Multi-agent interactions can produce noisy travel times if sample sizes are too small.	Low challenge. Automate paired t-tests, one-way ANOVA, and boxplot generation from recorded logs.

3.2 Workflow and Algorithm Choices

Figure 2 shows how the methodology steps depend on one another. The main critical path runs sequentially from environment setup (M0) and baseline simulation (M1) through the vision-LiDAR tracker (M2), the predictive MPPI critic (M3), and final statistical evaluation (M5).

Because the automated testing harness (M4) only depends on the simulation world from M1, it can be built in parallel alongside M2 and M3 to save time. Figure 2 also includes a fallback path (F1): if writing or linking the native C++ MPPI critic plugin takes longer than expected, the tracked velocity vectors from M2 can be injected into the 2D costmap as velocity-stretched obstacle zones so that comparative testing in M5 can still go ahead.

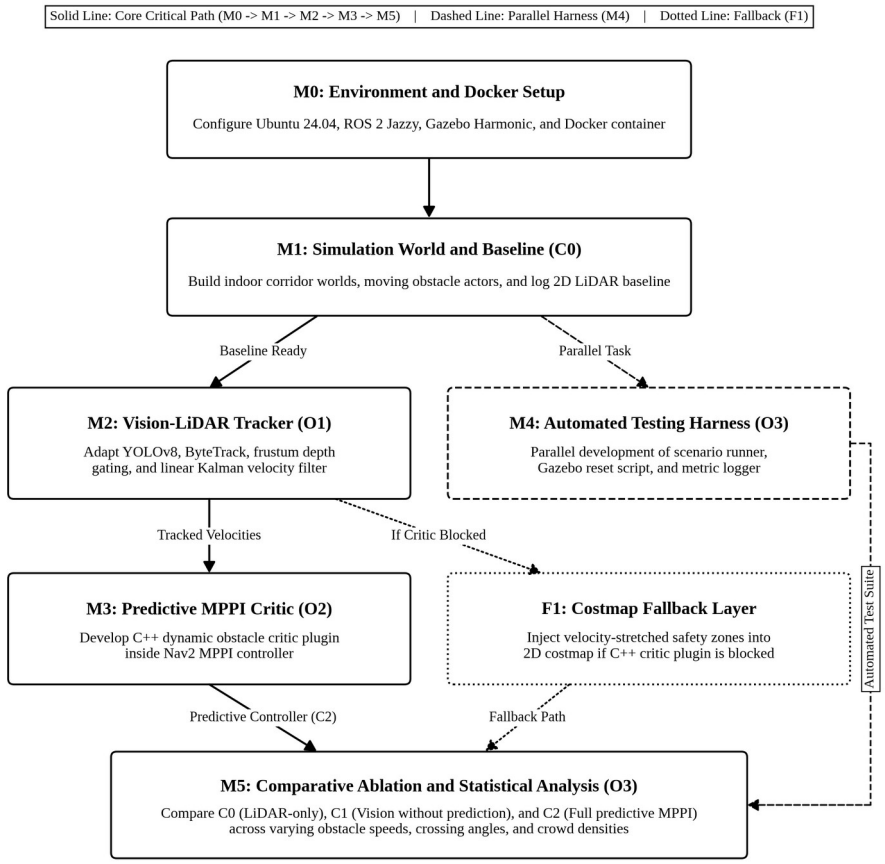


Figure 2. Top-down methodology workflow showing the main critical path (solid), parallel testing harness (dashed), and fallback option (dotted).

Several practical trade-offs guided the choice of algorithms for each subsystem. For obstacle detection and depth estimation (O1), I considered 2D laser-only clustering, DeepSORT, and full 3D point cloud networks. Pure 2D laser clustering was ruled out because a single-plane scan splits walking legs into separate clusters and cannot tell a person apart from a static pillar, while 3D point cloud networks need dense 3D LiDAR data and heavy GPU memory. Instead, running YOLOv8n with ByteTrack [8] keeps stable track IDs in 2D image space on the CPU, and projecting the 2D laser scan into each bounding box frustum [6] gives accurate floor distances using a simple median filter.

For obstacle motion prediction (O1/O2), I compared a Constant-Velocity (CV) linear Kalman filter [7] against the Social Force Model [9], Interactive Gaussian Processes [4], and deep trajectory transformers. In indoor corridors, pedestrians and pushcarts generally walk or roll at a steady speed and direction over short 1 to 2 s windows [9]. Running iterative social force simulations or neural networks across 2,000 MPPI trajectory rollouts at 20 Hz would be far too slow on a standard CPU, whereas linear constant-velocity extrapolation takes less than 0.2 ms.

Finally, for local path planning (O2), I chose to extend the Nav2 MPPI controller [5], [10] rather than using the Dynamic Window Approach (DWA), Timed Elastic Band (TEB), or post-filter Control Barrier Functions [11]. DWA only samples constant-velocity arcs and frequently

freezes when an obstacle crosses ahead, while TEB often oscillates between left and right passes in multi-agent intersections. By adding a dynamic obstacle critic directly inside MPPI, the planner can evaluate future obstacle positions across all 2,000 sampled trajectories and smoothly adjust the robot's speed to pass behind moving traffic.

4. Originality

4.1 Project Novelty and Comparison

Table 3 compares the proposed framework against standard industrial 2D LiDAR AMRs, non-predictive vision-LiDAR trackers, and recent high-compute GPU MPPI methods.

Table 3. Comparison of the proposed system against standard 2D LiDAR navigation, non-predictive trackers, and high-compute GPU MPPI methods.

Feature	Standard Industrial AMR (Nav2 + 2D LiDAR [2])	Vision-LiDAR Trackers (Without Prediction)	High-Compute GPU MPPI Variants [11]-[13]	Proposed Framework (This Project)
Sensor Suite	Planar 2D LiDAR only (around $400)	2D/3D LiDAR + RGB or depth camera ($500-$4,000)	Dense 3D LiDAR or depth point clouds (> $3,000)	2D LiDAR + monocular RGB camera (around $450)
Obstacle Representation	Static 2D costmap cells fixed at current time (t = 0)	Tracked bounding boxes written into static 2D costmap	Unsegmented 3D point clouds [12] or reactive barriers [11]	Static walls in 2D costmap + moving velocity tracks
Trajectory Evaluation	Static costmap critics only; freezes in cross-traffic	Static clearance inflation with raycast clearing delay	GPU signed-distance fields [12] or GPU clustering [13]	Lightweight C++ MPPI critic with 2.0 s motion prediction
Compute Requirement	Standard onboard CPU	Onboard CPU + edge GPU	Dedicated high-compute CUDA GPU (not edge-optimised)	Standard CPU for planning (< 0.3 ms) + light 2D detector
Evaluation Setup	Static obstacle avoidance tests	Isolated perception tracking metrics only	End-to-end comparison against older planners	Controlled 3-way ablation (C0 vs. C1 vs. C2)

In current industrial practice, brownfield AMRs either rely on a basic 2D LiDAR with static costmaps (which causes the robot to stop and wait whenever someone walks across its path) or require an expensive upgrade to 3D LiDARs and high-end onboard GPUs [1], [2]. As shown in Table 3, the originality of this project lies in bridging that gap with a practical, low-cost software and sensor setup. By pairing a standard 2D laser scanner with a single monocular camera, the system gets reliable visual class detection from the camera and accurate metric distances from the laser scanner without needing a 3D LiDAR.

On the planning side, all 11 default critics shipped with the ROS 2 Nav2 MPPI controller only check candidate trajectories against the static 2D costmap at the current instant [10]. Recent research papers such as SE-MPPI [11], EXACT-MPPI [12], and CSC-MPPI [13] have extended MPPI for complex environments, but they rely on heavy GPU computation (such as CUDA signed-distance fields or GPU DBSCAN clustering) that is not optimised for standard industrial robot computers, or act only as reactive safety filters on the final command [11]. Instead, this project implements a lightweight C++ dynamic obstacle critic plugin that projects tracked velocity vectors over a 2.0 s horizon on a standard CPU in under 0.3 ms. Furthermore, running a 3-way ablation study across C0 (LiDAR-only), C1 (visual detection without motion prediction), and C2 (full predictive planning) clearly separates the benefit of visual detection from the benefit of motion prediction.

4.2 Project Limitations

There are three main practical limitations in this project that directly shape the specification in Table 1. First, because the project is 100% simulation-based (0% hardware), virtual Gazebo actors do not capture all real-world camera lighting issues or uneven human leg motion; to account for this, Table 1 defines both a realistic minimum pass threshold (85% track matching) and a target threshold (95%). Second, because the monocular camera has an 80° horizontal field of view compared to the 360° 2D LiDAR, velocity tracking is only active inside the front 80° camera cone. Outside this cone, the robot falls back to the 360° 2D LiDAR static costmap so that obstacles approaching from the sides or rear still trigger safe collision avoidance. Third, the constant-velocity motion model assumes obstacles move in a straight line over short windows and will lag if someone suddenly changes direction; therefore, predictions are capped at 2.0 s with a growing safety margin over time.

5. Risk Management

Table 4 lists the four main technical and project management risks, scored by Likelihood (L, 1 to 5), Impact (I, 1 to 5), and overall Risk Score (L × I), along with practical mitigation steps.

Table 4. Project risk register showing likelihood (L), impact (I), risk score, system effect, and mitigation strategies.

ID	Risk Description	L	I	Score	Effect on Project	Mitigation & Fallback Plan
R1	Custom C++ MPPI critic plugin runs into Nav2 integration issues or exceeds the 50 ms (20 Hz) CPU time budget (O2).	4	5	20	Controller update rate drops below 20 Hz or plugin fails to load in Nav2.	Use vectorized Eigen math and cap lookahead at 2.0 s. If blocked, use Fallback F1 by writing velocity-stretched safety zones into the 2D costmap.
R2	Camera-LiDAR projection errors or laser rays hitting background walls behind legs corrupt velocity estimates (O1).	3	4	12	Noisy obstacle positions cause jumpy velocity vectors and poor predictions.	Read exact sensor transforms from TF2, take the median laser depth inside each 2D box, and reject sudden position jumps in the Kalman filter.
R3	Gazebo simulation instability or scope creep delays the 3-way batch benchmark runs (O3).	3	4	12	Incomplete statistical data across C0, C1, and C2 or simulator crashes.	Pin all ROS 2 and Gazebo packages in Docker, add watchdog auto-restarts to the test script, and only run stretch goals after core scenarios finish.
R4	Visual occlusion when multiple obstacles cross each other causes track ID switching (O1/O2).	3	3	9	Brief loss of velocity tracking during multi-person intersection crossings.	Use ByteTrack two-stage association [8] to keep low-confidence occluded boxes, and coast Kalman filter states for up to 1.0 s before dropping tracks.

6. Planned Work Schedule

Table 5 shows the 30-week Gantt chart split across the Autumn Semester (Weeks 1 to 15, shaded in blue-grey) and Spring Semester (Weeks 16 to 30, shaded in green-grey), based on a total workload of 450 hours (15 hours per week).

Table 5. Project Gantt chart across the Autumn Semester (Weeks 1-15) and Spring Semester (Weeks 16-30), including progress reviews (PR1-PR5).

Project Task	Autumn Semester (Weeks 1-15)							Spring Semester (Weeks 16-30)						
Activity / Weeks	**W1-2**	**W3-4**	**W5-6**	**W7-8**	**W9-10**	**W11-12**	**W13-15**	**W16-18**	**W19-20**	**W21-22**	**W23-24**	**W25-26**	**W27-28**	**W29-30**
1. Setup environment & Docker (W1-W3, 45 h)		**PR1**												
2. Simulate world, actors & C0 baseline (W3-W6, 45 h)			**PR2**											
3. Prepare vision-LiDAR tracker (O1) (W7-W12, 90 h)						**PR3**								
4. Setup automated testing harness (O3) (W8-W18, 45 h)														
5. Develop predictive MPPI critic (O2) (W13-W18, 75 h)								**PR4**						
6. Run comparative simulation benchmarks (W19-W24, 60 h)											**PR5**			
7. Analyze results & statistical metrics (W22-W26, 30 h)														
8. Write final thesis & prepare video demos (W25-W30, 60 h)														

In the Autumn Semester, Weeks 1 to 3 (45 h) cover setting up Ubuntu 24.04, ROS 2 Jazzy, Gazebo Harmonic, and the Docker container, concluding with the first supervisory progress review (PR1). Weeks 3 to 6 (45 h) focus on building the indoor corridor worlds, scripting the moving obstacle actors, and logging the C0 2D LiDAR-only baseline (PR2 at Week 6). Weeks 7 to 12 (90 h) are dedicated to building and verifying the YOLOv8, ByteTrack, frustum depth gating, and Kalman filter tracking package for Objective O1 (PR3 at Week 12). Starting in Week 8 and running in parallel through Week 18 (45 h), the automated Python scenario runner and metric logger are developed so that each module can be tested as soon as it is ready.

Spanning late Autumn and early Spring (Weeks 13 to 18, 75 h), the custom C++ dynamic obstacle MPPI critic plugin (Objective O2) and the non-predictive C1 baseline are implemented and tuned to run at 20 Hz (PR4 at Week 18, which also serves as the decision point for activating Fallback F1 if needed). In the Spring Semester, Weeks 19 to 24 (60 h) execute the batch simulation trials comparing C0, C1, and C2 across all obstacle speeds and crowd densities (PR5 at Week 24). Finally, Weeks 22 to 26 (30 h) cover the statistical analysis (paired t-tests, ANOVA, and performance plots), overlapping with Weeks 25 to 30 (60 h) to write the final MEng thesis and record the simulation demonstration videos.

References

[1] International Federation of Robotics, "Service robots see global growth boom," in *World Robotics 2025 – Service Robots*, Frankfurt, Germany: IFR, Oct. 7, 2025. [Online]. Available: https://ifr.org/ifr-press-releases/news/service-robots-see-global-growth-boom. Accessed: Nov. 4, 2026.

[2] S. Macenski, F. Martín, R. White, and J. Ginés Clavero, "The Marathon 2: A navigation system," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Las Vegas, NV, USA, Oct. 2020, pp. 2718–2725, doi: 10.1109/IROS45743.2020.9341207.

[3] S. Macenski, T. Moore, D. V. Lu, A. Merzlyakov, and M. Ferguson, "From the desks of ROS maintainers: A survey of modern & capable mobile robotics algorithms in the Robot Operating System 2," *Robot. Auton. Syst.*, vol. 168, Oct. 2023, Art. no. 104493, doi: 10.1016/j.robot.2023.104493.

[4] P. Trautman and A. Krause, "Unfreezing the robot: Navigation in dense, interacting crowds," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Taipei, Taiwan, Oct. 2010, pp. 797–803, doi: 10.1109/IROS.2010.5654369.

[5] G. Williams, N. Wagener, B. Goldfain, P. Drews, J. M. Rehg, B. Boots, and E. A. Theodorou, "Information theoretic MPC for model-based reinforcement learning," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, Singapore, May–Jun. 2017, pp. 1714–1721, doi: 10.1109/ICRA.2017.7989202.

[6] Q. Zhang and R. Pless, "Extrinsic calibration of a camera and laser range finder (improves camera calibration)," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Sendai, Japan, Sep.–Oct. 2004, vol. 3, pp. 2301–2306, doi: 10.1109/IROS.2004.1389752.

[7] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. Cambridge, MA, USA: MIT Press, 2005.

[8] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang, "ByteTrack: Multi-object tracking by associating every detection box," in *Computer Vision – ECCV 2022* (Lecture Notes in Computer Science), vol. 13682, Cham, Switzerland: Springer, Oct. 2022, pp. 1–21, doi: 10.1007/978-3-031-20047-2_1.

[9] D. Helbing and P. Molnár, "Social force model for pedestrian dynamics," *Phys. Rev. E*, vol. 51, no. 5, pp. 4282–4286, May 1995, doi: 10.1103/PhysRevE.51.4282.

[10] S. Macenski and A. Budyakov, "MPPI Controller (nav2_mppi_controller)," *Nav2 Documentation*, Open Navigation LLC, 2024. [Online]. Available: https://docs.nav2.org/configuration/packages/configuring-mppic.html. Accessed: Nov. 4, 2026.

[11] J. Kang, "SE-MPPI: Coordinating online local-minima escape with control-barrier-function safety in a Nav2-native controller," Tech. Preprint / GitHub Repository, 2026. [Online]. Available: https://github.com/kjungmo/se-mppi-nav2. Accessed: Nov. 4, 2026.

[12] C. Peng, Z. Ge, W. Lu, H. Gao, S. Vougioukas, and P. Wei, "EXACT-MPPI: Exact signed-distance navigation for arbitrary-footprint robots from point clouds via path integral control," 2026, *arXiv:2605.29663*.

[13] L. Park, K. Jang, and S. Kim, "CSC-MPPI: A novel constrained MPPI framework with DBSCAN for reliable obstacle avoidance," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Hangzhou, China, Oct. 2025, pp. 19645–19652, doi: 10.1109/IROS60139.2025.11246839.

Appendix A: Supervisor-Provided Project Specification

Project Title: Vision-Augmented Mobile Robot Navigation in Dynamic Environments
Project Code: SD-MEng-26-01 | **Supervisor:** Dr Shalini Darmaraju | **Moderator:** Ir Dr Daniel Pu | **Date:** 19 October 2026

A.1 Project Description

Autonomous mobile robots need to navigate safely and efficiently in environments containing moving obstacles. Planar range sensors provide distance measurements, while cameras provide visual information that can help identify relevant obstacles. Combining these inputs with obstacle motion estimation can support navigation decisions when obstacles cross or approach the robot's planned route.

This project aims to develop a simulation-based navigation framework that combines camera information with planar range measurements for obstacle tracking and motion-aware navigation. A mobile robot equipped with simulated sensors will operate in a known indoor environment containing selected types of moving obstacles. Existing visual detection and tracking methods will be adapted and combined with range measurements to estimate obstacle positions and velocities. A simple motion model will predict their short-term positions, which will be incorporated into local path planning and collision avoidance using an existing navigation framework.

The proposed system will be evaluated against range-sensor-based navigation, with additional comparisons to distinguish the effects of visual information and motion prediction. Experiments will vary obstacle speeds, movement patterns and density. Evaluation will assess tracking accuracy, navigation success, collisions, travel time, waiting time and computational efficiency. All development and evaluation will be conducted in simulation, with images and range measurements generated by simulated sensors.

A.2 Target Sustainable Development Goals (SDGs)

- **SDG 9:** Industry, Innovation and Infrastructure
- **SDG 8:** Decent Work and Economic Growth

A.3 Project Objectives

- 1. To review existing methods for vision-augmented mobile robot navigation, camera and range-sensor integration, obstacle tracking and motion prediction.
- 2. To develop an indoor robot simulation and adapt existing detection and tracking methods to combine simulated camera and planar range measurements for obstacle position and velocity estimation.
- 3. To develop a navigation framework that incorporates short-term obstacle motion predictions into local path planning and collision avoidance.
- 4. To evaluate the proposed framework against range-sensor-based navigation under varying obstacle movements and densities, assessing tracking accuracy, navigation performance and computational efficiency.

A.4 Project Deliverables & Resource Breakdown

- 1. A literature review of vision-augmented mobile robot navigation, camera and range-sensor integration, obstacle tracking and motion prediction methods.
- 2. A configurable indoor robot simulation and an implemented obstacle detection and tracking system using simulated camera and range measurements, with documented source code.
- 3. A navigation prototype integrating short-term obstacle motion prediction with local path planning and collision avoidance.
- 4. Comparative experimental results and analysis of tracking accuracy, navigation success, collisions, travel time, waiting time and computation time, supported by reproducible simulation scenarios and video demonstrations.
- 5. A final project report documenting the literature review, methodology, system development, evaluation results and conclusions.

Resource Ratio: Hardware 0% | Software 60% | Research/Investigation/Design 40%. **Facilities:** Computer workstation only (Ubuntu Linux, ROS 2, Nav2, Gazebo, Python/C++, NumPy, Matplotlib).